

AMDP 練習 4：SQLScript 集合處理——多表 JOIN / 聚合 / CTE

Lecture

am01/am02 的 **SELECT** 都只查單一張表；am03 示範了命令式的逐筆處理。這題回到宣告式風格，但把場景升級成多表 **JOIN + 聚合 + CTE (Common Table Expression, WITH 子句)**——這才是 SQLScript / Code-to-Data 真正發揮威力的地方：把「先查一張表、再查另一張表、再到應用層兜起來」這種多階段運算，濃縮成資料庫引擎一次執行的單一敘述。

這題的業務場景：計算每條航線 (**carrid + connid**) 的**載客率** (load factor：已訂位座位數 / 總座位數)。這個需求天生要橫跨兩張表——**SFLIGHT** (每個航班每個日期的座位數) 跟 **SCARR** (航空公司名稱，純粹是為了讓輸出好讀)，而且要先對 **SFLIGHT** 依航線分組加總，才能算比例。

WITH ... AS (...) 就是 **CTE**：把一段子查詢取一個名字 (這題是 **route_totals**)，後面的主查詢可以像引用一張真實的表一樣引用它。這跟「先把子查詢結果存進一個 Table Variable，再對這個變數做下一步查詢」的效果類似，但寫成 CTE 語意更清楚——「先算出 A，再拿 A 去算 B」的兩階段邏輯，一次敘述、一次資料庫執行就搞定，不需要像 ABAP 那樣「查一次、存進 Internal Table、再迴圈或再查一次」。

學習目標

- 會用 **WITH <名稱> AS (子查詢)** 宣告一個 CTE，並在後續的主查詢裡把它當一張表引用 (**FROM <名稱>**)
- 會寫多表 **JOIN ... ON ...**，並且理解 SQLScript 裡 JOIN 條件可以明寫 **MANDT** 欄位 (**ON c.mandt = rt.mandt**)——這跟 REST 課程 [.claude/rules/sap-adt-mcp.md](#) 第 10 節記載的「Open SQL 的 JOIN ON 條件不能明寫 MANDT」正好相反，因為 SQLScript 沒有 ABAP 編譯器那層自動的 client 處理，所有跟 Client 有關的邏輯 (包含 JOIN 條件) 都要自己明確寫出來
- 會寫 **GROUP BY** 搭配聚合函數 (**COUNT(*)**、**SUM(...)**) 算出每組的統計
- 認識 **CAST(... AS DECIMAL(5,1))** 這種型別轉換語法，讓計算出來的比例有正確的小數位數
- 會用 **UNION ALL** 把兩段條件不同、但欄位形狀一樣的 **SELECT** 合併成一個結果集；認識幾個常用的 SQLScript 字串函數 (**CONCAT**、**UPPER**) 跟日期函數 (**DAYS_BETWEEN**、**CURRENT_DATE**)

事前準備

ADT 端物件已由課程準備好 (**\$TMP**)：

- ZCL_AM04_ROUTE_LOAD**——AMDP 類別，含兩個方法：**get_route_load_factor** 用 CTE 先依航線彙總 **SFLIGHT**，再 JOIN **SCARR** 補上航空公司名稱，算出每條航線的載客率；**get_flight_timeline** 用 **UNION ALL + 字串/日期函數** 示範另一種集合處理手法
- ZR_AM04_DEMO / ZR_AM04_TIMELINE**——demo 程式，分別呼叫上述兩個方法，用 **WRITE** 印出結果

題目需求 (對照已建好的答案物件，含實測踩坑過程)

- ZCL_AM04_ROUTE_LOAD** 的簽章與 SQLScript 本體：

```
``abap CLASS zcl_am04_route_load DEFINITION PUBLIC FINAL CREATE PUBLIC. PUBLIC SECTION. INTERFACES
if_amdp_marker_hdb.
```

```
TYPES: BEGIN OF ty_route_load, carrid TYPE s_carr_id, carrname TYPE s_carrname, connid TYPE s_conn_id,
flight_cnt TYPE i, seats_occ TYPE i, seats_max TYPE i, load_pct TYPE p LENGTH 5 DECIMALS 1, END OF
ty_route_load, tt_route_load TYPE STANDARD TABLE OF ty_route_load WITH EMPTY KEY.
```

```
CLASS-METHODS get_route_load_factor IMPORTING VALUE(iv_mandt) TYPE mandt EXPORTING
VALUE(et_routes) TYPE tt_route_load. ENDCLASS.
```

```
CLASS zcl_am04_route_load IMPLEMENTATION.
```

```
METHOD get_route_load_factor BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-
ONLY USING scarr sflight.
```

```
et_routes = WITH route_totals AS ( SELECT f.mandt, f.carrid, f.connid, COUNT(*) AS flight_cnt, SUM(f.seatsocc)
AS seats_occ, SUM(f.seatsmax) AS seats_max FROM sflight AS f WHERE f.mandt = :iv_mandt GROUP BY
f.mandt, f.carrid, f.connid ) SELECT rt.carrid, c.carrname, rt.connid, rt.flight_cnt, rt.seats_occ, rt.seats_max,
CAST(rt.seats_occ * 100.0 / rt.seats_max AS DECIMAL(5,1)) AS load_pct FROM route_totals AS rt JOIN scarr AS
c ON c.mandt = rt.mandt AND c.carrid = rt.carrid ORDER BY rt.carrid, rt.connid;
```

```
ENDMETHOD.
```

```
ENDCLASS. ``
```

1. 實測踩坑：USING scarr, sflight (逗號分隔) 啟用失敗：

第一版把 USING 子句寫成跟 ABAP 一般列舉多個值時的直覺習慣一樣，用逗號分隔：

```
``abap " 第一版，啟用失敗 USING scarr, sflight. ``
```

啟用時 ABAP 編譯器 (不是 HANA/SQLScript 那層，這是編譯 **AMDP** 方法簽章本身時的錯誤) 回報：Comma without preceding colon (after METHOD ?)。AMDP 方法宣告裡的 USING 子句，多個資料庫物件是用空白分隔，不是逗號——USING scarr sflight. 才是合法語法。這跟 am01/02/03 都只有單一 USING <一張表> 沒機會踩到，這題第一次用到兩張表才發現。

1. 這題也再次印證/延伸 am01 的 Client 過濾教訓，但方向相反：.claude/rules/sap-adt-mcp.md 第 10 節記載過，Open SQL 的 JOIN ... ON 條件不能明寫 MANDT 欄位 (會報語法錯誤 The client field MANDT cannot be specified in the ON condition，client 比對由編譯器自動處理)；但這題的 SQLScript JOIN scarr AS c ON c.mandt = rt.mandt AND c.carrid = rt.carrid 明寫 mandt 完全沒問題，而且是必要的——因為 SQLScript 沒有 ABAP 編譯器那層自動處理 Client 的機制 (呼應 am01 的核心教訓)，JOIN 條件如果沒有明確比對 mandt，會產生跨 Client 的錯誤配對 (例如 Client 100 的 SFLIGHT 統計結果，被拿去跟 Client 130 的 SCARR 名稱配對)。同一個「JOIN 條件要不要寫 MANDT」的問題，Open SQL 跟 SQLScript 給出完全相反的答案，原因都指向同一件事：Client 處理在哪一層被自動化了。

1. ZR_AM04_DEMO 呼叫端：

```
``abap REPORT zr_am04_demo.
```

```
zcl_am04_route_load=>get_route_load_factor( EXPORTING iv_mandt = sy-mandt IMPORTING et_routes = DATA(lt_routes) ).
```

```
LOOP AT lt_routes ASSIGNING FIELD-SYMBOL(<ls_route>). WRITE: / <ls_route>-carrid, <ls_route>-carrname, <ls_route>-connid, <ls_route>-flight_cnt, <ls_route>-seats_occ, <ls_route>-seats_max, <ls_route>-load_pct. ENDLOOP. ``
```

預期輸出 (實測畫面節錄)

AA	American Airlines	0017	12	2,880	4,620	62.3
AA	American Airlines	0064	13	2,832	4,290	66.0
...						
LH	Lufthansa	0400	16	3,220	5,350	60.1
LH	Lufthansa	0401	15	2,716	3,900	69.6
LH	Lufthansa	0402	15	5,148	7,125	72.2
...						
UA	United Airlines	3517	15	3,932	5,775	68.0

各航線的載客率落在 60%~72% 之間，且每個 **carrid/carrname** 都正確配對 (**SCARR** 的 JOIN 生效)、每條航線的 **flight_cnt** 跟 am01/am02 之前看到的明細筆數對得上——這代表 CTE 的分組彙總跟最後的 JOIN 都正確執行。

延伸範例：UNION ALL 與字串 / 日期函數

除了 CTE，SQLScript 集合處理另一個常用招式是 **UNION ALL**——把兩段條件不同、但欄位形狀一樣的查詢結果合併成一個結果集。這題再示範一個方法 **get_flight_timeline**，把某家航空公司的航班分成「已經飛過」 (**PAST**) 跟「還沒飛」 (**UPCOMING**) 兩類，順便秀一下常用的字串跟日期函數：

```
CLASS-METHODS get_flight_timeline
IMPORTING
    VALUE(iv_mandt)    TYPE mandt
    VALUE(iv_carrid)    TYPE s_carr_id
EXPORTING
    VALUE(et_timeline) TYPE tt_flight_timeline.
```

```
METHOD get_flight_timeline BY DATABASE PROCEDURE
FOR HDB LANGUAGE SQLSCRIPT
OPTIONS READ-ONLY
USING scarr sflight.

et_timeline =
    SELECT f.carrid,
           CONCAT(CONCAT(f.carrid, '-'), f.connid) AS route_code,
           UPPER(c.carrname) AS carrname_up,
           f.fldate,
           'PAST' AS flight_status,
```

```

        DAYS_BETWEEN(f.fldate, CURRENT_DATE) AS days_diff
FROM sflight AS f
JOIN scarr AS c ON c.mandt = f.mandt AND c.carrid = f.carrid
WHERE f.mandt = :iv_mandt AND f.carrid = :iv_carrid AND f.fldate < CURRENT_DATE
UNION ALL
SELECT f.carrid,
       CONCAT(CONCAT(f.carrid, '-'), f.connid) AS route_code,
       UPPER(c.carrname) AS carrname_up,
       f.fldate,
       'UPCOMING' AS flight_status,
       DAYS_BETWEEN(CURRENT_DATE, f.fldate) AS days_diff
FROM sflight AS f
JOIN scarr AS c ON c.mandt = f.mandt AND c.carrid = f.carrid
WHERE f.mandt = :iv_mandt AND f.carrid = :iv_carrid AND f.fldate >= CURRENT_DATE
ORDER BY fldate;

ENDMETHOD.

```

幾個重點：

- **UNION ALL** 要求前後兩段 **SELECT** 的欄位數量、順序、型別都要一致——這題兩段都是 **carrid**, **route_code**, **carrname_up**, **fldate**, **flight_status**, **days_diff** 六欄，只是 **flight_status** 常數值跟 **days_diff** 的算法不同（**PAST** 段是 **fldate** 到 **CURRENT_DATE** 的天數，**UPCOMING** 段是反過來），**ORDER BY** 寫在整個 **UNION ALL** 之後，對合併後的整體結果排序，不是各自排各自的
- **UNION vs UNION ALL**：這題用 **UNION ALL**，因為兩段 **WHERE** 條件（**fldate < CURRENT_DATE** 跟 **fldate >= CURRENT_DATE**）天生互斥、不會有重複列，**UNION ALL** 直接合併、不做去重，效能比 **UNION**（會額外做一次去重運算）好；如果兩段查詢真的可能撈出重複列、又不想要重複，才需要 **UNION**
- **CONCAT** 一次只能接兩個參數：這題要把 **carrid** / - / **connid** 三段接成一個字串（如 **LH-0400**），要寫成 **CONCAT(CONCAT(f.carrid, '-'), f.connid)** 巢狀兩層，不能一次塞三個參數進去
- **DAYS_BETWEEN(date1, date2)** 的結果是 **date2 - date1**（天數差）：**PAST** 段用 **DAYS_BETWEEN(f.fldate, CURRENT_DATE)**（過去日期在前）算出正數的「已經過了幾天」；**UPCOMING** 段用 **DAYS_BETWEEN(CURRENT_DATE, f.fldate)**（今天在前）算出正數的「還有幾天」——兩段刻意把日期參數順序對調，才能讓兩種情境都顯示成正數，不用另外處理負號

實測驗證：ZR_AM04_TIMELINE 呼叫 p_carrid = 'LH'，回傳 76 筆（跟 am01/am03 看到的 LH 總航班數一致），最舊的一筆 2018/09/30 顯示 **PAST** 且 **days_diff** 約 2849 天，唯一一筆未來日期 2027/01/01 顯示 **UPCOMING** 且 **days_diff** = 166——**UNION ALL** 沒有漏掉或重複任何一筆，**CONCAT** / **UPPER** / **DAYS_BETWEEN** 算出來的值都正確。

團隊實務備註

- **USING** 子句的分隔符號是空白不是逗號：這是 AMDP 方法簽章語法（ABAP 編譯器負責解析），不是 SQLScript 本體的語法，錯誤訊息也是 ABAP 端的訊息（**Comma without preceding colon**），跟前幾題遇到的「SQLSCRIPT: sql syntax error」（HANA 編譯器訊息）是不同來源，遇到編譯錯誤時看訊息開頭能幫助判斷問題出在 AMDP 簽章層還是 SQLScript 本體層
- **Open SQL vs SQLScript** 對「**JOIN** 條件能不能寫 **MANDT**」的規則完全相反，根源都是「**Client** 處理自動化在哪一層」：這是這門課到目前為止最值得記住的一組對照——寫 Open SQL 時不用管 Client、也

不能在 JOIN 條件裡管；寫 SQLScript 時完全要自己管，包含 JOIN 條件

- **CTE (WITH ... AS (...))** 只在這一次查詢裡有效，不是建立一個永久的資料庫物件，跟建一個實體的 View 不一樣——如果同一個 AMDP 方法後面還有其他查詢想重用 `route_totals` 這個中間結果，一樣要在需要的地方重新寫 **WITH** 或改成先指派給一個 Table Variable (`lt_route_totals = SELECT ...;`) 再重複引用
- **CAST(... AS DECIMAL(5,1))** 是 HANA SQL 的型別轉換語法，如果拿掉這個 **CAST**，`seats_occ * 100.0 / seats_max` 算出來的除法結果型別可能跟 ABAP 端 `TYPE p LENGTH 5 DECIMALS 1` 對不上，導致啟用或執行時出現型別轉換的警告/錯誤——遇到 AMDP 的數值計算結果要匯出給 ABAP 端，養成習慣明確 **CAST** 成跟 ABAP 端型別相容的 SQL 型別，比依賴自動轉換可靠
- **UNION ALL / CONCAT / UPPER / DAYS_BETWEEN** 這幾個一次就啟用成功、沒有踩坑——這題前面 (CTE/JOIN/USING 分隔符號) 踩過的坑都是「AMDP 簽章規則」或「SQLScript 特有語法」層級的問題，字串/日期函數跟 **UNION ALL** 本質上是很標準的 SQL 功能，多數關聯式資料庫都有類似語法，只要函數名稱查對 (**CONCAT/UPPER/DAYS_BETWEEN** 都是 HANA SQL 內建函數)，不太會有 AMDP 特有的額外限制

思考題

1. 如果拿掉 CTE，直接把 `route_totals` 的子查詢改寫成一段巢狀 `FROM (SELECT ... GROUP BY ...) AS rt JOIN scarr ...` (子查詢直接內嵌在 FROM 子句裡，不用 **WITH** 具名)，效果會不會一樣？CTE 相較於巢狀子查詢，可讀性上的優勢在哪裡？
2. 這題的 **JOIN** 是 **INNER JOIN** (預設) ——如果某個 Client 的 `SCARR` 缺了某個 `carrid` (理論上不該發生，但假設資料有缺)，這條航線會不會出現在 `et_routes` 裡？如果想要「即使 `SCARR` 缺資料，航線統計還是要顯示，只是 `carrname` 是空的」，**JOIN** 該怎麼改？
3. 如果想再擴充這題，加入第三張表 `SPFLI` (航線基本資料，例如起訖城市)，**USING** 子句、CTE、**JOIN** 分別要怎麼調整？(提示：**USING** 子句列出所有用到的表，空白分隔；再加一個 `JOIN spfli AS s ON s.mandt = rt.mandt AND s.carrid = rt.carrid AND s.connid = rt.connid`)
4. `get_flight_timeline` 用兩段 **WHERE** 條件 (`fldate < CURRENT_DATE` 跟 `fldate >= CURRENT_DATE`) 確保 **UNION ALL** 的兩段不會重複——如果不小心把第二段也寫成 `fldate <= CURRENT_DATE` (多了一個等號，兩段條件在 `fldate = CURRENT_DATE` 這天重疊)，**UNION ALL** 會不會把當天的航班算兩次？如果換成 **UNION** (不加 **ALL**) 呢？

答案

見 `zcl_am04_route_load.clas.abap` (含 `get_route_load_factor` / `get_flight_timeline` 兩個方法)、`zr_am04_demo.prog.abap`、`zr_am04_timeline.prog.abap` (SAP 端物件 `ZCL_AM04_ROUTE_LOAD` / `ZR_AM04_DEMO` / `ZR_AM04_TIMELINE`，套件 `$TMP`)。